Daily Scholar Papers Report — 2026-08-29

2026-08-29

Daily Scholar Papers Report — 2026-08-29

Download PDF

Window covered: 2026-08-28 → 2026-08-29 (Google Scholar alerts + user-curated self-emails, last 24 h)

Executive Summary

One alert thread, four candidates, three papers through screening — and two of the three are asking the same question from opposite ends of the same pipeline: **when you let a language model into a security-critical system, where exactly does the trust boundary go?**

The sharpest answer comes from an ASE ’26 paper on concolic execution. Its premise is that object-oriented path exploration stalls not on arithmetic but on the heap — nullness, aliasing, runtime types, field reachability — because satisfying those conditions means *building an object graph*, not computing a valuation. So the engine splits each path condition in two, hands the primitive half to an SMT solver, and hands the heap half to a solver that asks an LLM to propose a concrete heap model plus the Java code that constructs it. The LLM is untrusted and stays untrusted: four ordered checks — schema, constraint satisfaction, `javac` compilation, and a Soot-based heap-realization analysis — must all pass before anything is installed, and any inconclusive fact counts as failure. Across **6,750 APIs from ten Java libraries**, paths explored rise from **15,954 to 28,757 (+80.2 %)** and concrete valuations from **13,295 to 23,248 (+74.9 %)**. Against an LLM-assisted baseline it covers **15.5 % more branches with 98.8 % fewer tokens and 96.9 % less money** — $2.51 against $80.75.

The number that justifies the whole architecture is the one from the verifier: **13,879 candidate models were rejected**, and bounded refinement recovered **9,267 of them (66.77 %)**. More than half the rejections are plain compilation failures. An LLM proposing heap models is wrong most of the time — and it is still useful, because the check is cheap and total.

The second paper approaches the same boundary with no verifier available, and the result is visibly harder. It is a nine-year longitudinal study of **30,572 vulnerability reports across 16,802 packages, 10 languages and eight package managers, 2017 to late 2025**, and its measurement half is bleak in a precise way: vulnerable packages grow **91.5 % annually** against **26 %** for packages overall, while deployed fixes have sat flat at roughly **1,650 per year since 2022** — a fix-to-report ratio near **0.44**, and **over 7,000 unresolved vulnerabilities accumulated between 2022 and 2024** alone. That capacity ceiling is what motivates the second half: ten LLMs evaluated as CI/CD gatekeepers on real vulnerable/patched version diffs. The best balanced model reaches **62.8 % F1**. Nothing occupies the useful corner — one model gets 84.9 % recall at 45.8 % precision, another 90.1 % precision at 11.0 % recall, and the most reasoning-heavy one takes **50 seconds per sample**. The authors do not dress this up; they conclude that a staged pipeline is the only honest reading.

Read together, the pair suggests a rule worth stating plainly: *an LLM can be admitted into a security-critical pipeline exactly to the degree that a cheap, total verifier exists downstream of it.* Concolic execution has one — compile the code, run it, check the heap. Vulnerability triage does not, and pays 62.8 % F1 for the absence.

One further paper is carried at summary depth and labelled as such: a cross-corpus IoT-firmware benchmark whose contribution is the evaluation protocol rather than another detector. One candidate was screened out at Stage-1 as saturated; it is not named.

Outstanding: 2 · **Keep:** 1 · **Borderline High-Priority:** 0

Highlighted Papers

Title	Authors	Venue	Link
Verifier-in-the-Loop LLM Solving of Heap Constraints for Concolic Execution	S. Xia, L. Cheng, C. Dong, D. She, B. Wang, X. Peng, Z. Dong	ASE '26 — 41st IEEE/ACM Int. Conf. on Automated Software Engineering, Munich, Oct 2026	doi:10.1145/3832783.3834410 · author PDF
Vulnerability Evolution and the Promise of Automated Gatekeeping in Open-Source Software	S. A. Akhavani, B. Ousat, S. Uluagac, A. Kharraz	RAID 2026	author PDF · dataset
Cross-Corpus Evaluation of Generalizable Vulnerability Detection in IoT Firmware	S. H. Rumman, M. S. Islam, M. R. Rahman	arXiv preprint, Aug 2026 (cs.CR)	arXiv:2608.11492

Papers

1.1 · CONCOLIC EXECUTION · 13,879 LLM proposals rejected, 66.8% repaired — and +80.2% paths at 1/85th the token cost

Verifier-in-the-Loop LLM Solving of Heap Constraints for Concolic Execution

Authors: Shaoran Xia, Leyi Cheng, Caihua Dong, Dongdong She, Bo Wang, Xin Peng, Zhen Dong

Venue: ASE '26 — Proceedings of the 41st IEEE/ACM International Conference on Automated Software Engineering, October 12–16 2026, Munich, Germany. ACM, 13 pages.

Links. ACM DOI 10.1145/3832783.3834410 · author-hosted PDF. **License: CC BY 4.0** — the front matter states the work is licensed under a Creative Commons Attribution 4.0 International License. Full text read (13 pp., incl. appendix-level implementation detail). Artifact (source, subjects, scripts, logs, generated suites, Docker config) is stated to be on Figshare.

The bottleneck being named

Concolic execution accumulates path constraints and asks an SMT solver for inputs that drive execution down unexplored branches. For integers, booleans and floats this works. The paper's claim is that in object-oriented code the binding constraint is somewhere else entirely: branch feasibility turns on whether a reference is null, whether two variables alias, whether an `instanceof` test succeeds, whether a field dereference is realizable, and which concrete method a virtual call dispatches to. The authors name this class **heap constraints** and observe that they are *existential over heap structures* — satisfying one requires constructing a concrete object graph, not computing a valuation.

Two measurements establish that this is systematic rather than anecdotal. In a manual study of 60 randomly selected `commons-lang3` APIs, native JDart completed on **87.5 %** but reached full branch coverage on only **42.5 %**; for the **45 %** that terminated without full coverage, manual inspection identified unresolved heap constraints as the dominant recurring cause. At benchmark scale, across 6,750 APIs, native JDart explores **15,954** paths and produces **13,295** concrete valuations.

The running example is a simplified `canOverlay` from JFreeChart's rendering pipeline. Five guards must pass before the target path: two null checks, two `instanceof` tests, one reference disequality — and only then a primitive `getSeriesCount() > 0`. The asymmetry is the point. Once a receiver object exists, SMT answers the integer question trivially. What it cannot supply is *which* implementation of `Dataset` should back `ds1`, whether `baseline` points to a fresh object, and which expressions must alias.

The decomposition

For a selected target path with recorded path condition, COHEC decomposes

PC=PCprim∧PCheapPC = PC_{\text{prim}} \wedge PC_{\text{heap}}

where PCprimPC_{\text{prim}} ranges over primitive symbolic variables and PCheapPC_{\text{heap}} is a quantifier-free Boolean combination of heap atoms. Solving the first yields a valuation σprim\sigma_{\text{prim}}; solving the second yields a concrete heap state s=(Γ,ρ,H)s = (\Gamma, \rho, H), where Γ\Gamma records runtime types, ρ\rho maps variables to locations or `null`, and HH maps allocated locations to pairs (C,F)(C, F) of runtime class tag and finite field store. Composition is

σ=Compose(σprim,s)=s⊕σprim\sigma = \mathrm{Compose}(\sigma_{\text{prim}}, s) = s \oplus \sigma_{\text{prim}}

so that if σprim⊨PCprim\sigma_{\text{prim}} \models PC_{\text{prim}} and s⊨PCheaps \models PC_{\text{heap}}, then Compose(σprim,s)⊨PC\mathrm{Compose}(\sigma_{\text{prim}}, s) \models PC.

Four atomic forms carry the heap half — nullness `IsNull(p)`, aliasing `Alias(p,q)` and `Diseq(p,q)`, subtype `InstanceOf(p,T)`, and exact type `ExactType(p,D)` — over object expressions that are either root symbolic variables or bounded field paths such as `ds1.baseline`. Their satisfaction relation is given explicitly (Section 3.2):

s⊨IsNull(p)⇔[[p]]s=null,s⊨¬IsNull(p)⇔[[p]]s≠null,s⊨Alias(p,q)⇔[[p]]s=[[q]]s≠null,s⊨Diseq(p,q)⇔[[p]]s≠[[q]]s,s⊨InstanceOf
\begin{aligned} s &\models \mathrm{IsNull}(p) &&\iff [\![p]\!]_s = \mathrm{null},\\ s &\models \neg\mathrm{IsNull}(p) &&\iff [\![p]\!]_s \neq \mathrm{null},\\ s &\models \mathrm{Alias}(p,q) &&\iff [\![p]\!]_s = [\![q]\!]_s \neq \mathrm{null},\\ s &\models \mathrm{Diseq}(p,q) &&\iff [\![p]\!]_s \neq [\![q]\!]_s,\\ s &\models \mathrm{InstanceOf}(p,T) &&\iff [\![p]\!]_s = \ell \neq \mathrm{null} \wedge H(\ell) = (C,F) \wedge

C \leq_{CT} T,\\ s &\models \mathrm{ExactType}(p,D) &&\iff [\![p]\!]_s = \ell \neq \mathrm{null} \wedge H(\ell) = (D,F). \end{aligned}

For the running example this instantiates to a conjunction of six atoms: both datasets non-null, `ds1` and `ds1.baseline` satisfying the subtype tests, `ds1.baseline` non-null, and `ds1.baseline` distinct from `ds2`. The `getSeriesCount() > 0` guard is absent — it lives in PCprimPC_{\text{prim}}.

Worth noting for anyone building on this: the decomposition operates on the path condition **as recorded by the underlying executor**. COHEC does not recover aliasing or other heap facts that constraint collection dropped. The guarantee is relative to what JDart wrote down.

Three stages, then four checks

The Heap Constraint Solver produces what the paper calls, in its one numbered definition:

> **Definition 3.1 (Candidate Model).** A candidate model is a triple w=(β,ℋw,code)w = (\beta, \mathcal{H}_w, \texttt{code}), where β\beta maps each object expression occurring in PCheapPC_{\text{heap}} to a location or null, ℋw\mathcal{H}_w is a finite partial heap model, and `code` is initialization code intended to realize (β,ℋw)(\beta, \mathcal{H}_w).

The definition's last sentence is the whole design in miniature: no consistency or executability is assumed at this stage. Construction proceeds in three dependent stages — **runtime type assignment** (which concrete classes are admissible under declared types, subtype atoms and dispatch-induced exact-type atoms), **object graph construction** (which expressions are null, which alias, which objects are reused from the current heap versus allocated fresh, and what field links are required), and **initialization code synthesis** (an executable sequence using constructors, factories, builders, setters). Strings, lists and maps are initialized through public-API templates rather than naive field assignment, because naive allocation skips representation invariants.

Then the verifier, in fixed order, each check consuming what the previous admitted:

1. **Schema and context check** — parse against the candidate schema; reject out-of-scope references, unresolved identifiers, malformed entries.
2. **Constraint satisfaction check** — resolve every object expression through β\beta and ℋw\mathcal{H}_w, re-evaluate every atom and Boolean connective; the full formula must evaluate true.
3. **Compilation check** — invoke standard `javac` against the target program and its dependencies.
4. **Heap realization check** — analyze the compiled initialization code with **Soot**, compare the inferred post-state against (β,ℋw)(\beta, \mathcal{H}_w) projected onto the relevant object expressions. *If it cannot establish a required fact, it rejects.*

That last clause is the load-bearing one. Rejection on inconclusiveness is what makes the policy fail-closed rather than best-effort. The solver returns SAT only for a verified model; UNSAT only for deterministic contradictions such as incompatible exact-type requirements on one expression; everything else — generation failure, verifier rejection, budget exhaustion, SMT uncertainty — is UNKNOWN. The paper is explicit that "failure to generate a candidate or pass verification does not imply UNSAT."

One efficiency mechanism deserves separate mention because it is reusable outside this setting. **History retrieval**: many target paths diverge only after a common prefix, so before solving a new PCheapPC_{\text{heap}} the engine may retrieve accepted solving records from earlier paths sharing an execution prefix. These are used strictly as optional examples — they never constrain the current candidate and never relax acceptance. Efficiency on the untrusted side, soundness untouched on the trusted side.

The numbers

Subjects: **6,750 APIs** across `commons-lang3` 3.13.0, `guava` 32.1.2, `iCal4j` 3.2.11, `jfreechart` 1.5.4, `jgrapht` 1.3.1, `joda-time` 2.12.5, `threetenbp` 1.6.8, `Time4J` 5.9.1, `SIS-Utility` 1.3, `XChart` 3.8.7. Each API wrapped as a single-entry harness. **50 of 6,750** APIs skipped because the baseline timed out.

RQ1 — exploration against native JDart (Base) and two SV-COMP JDart variants:

Metric	Base	JDart-2020	JDart-2021	COHEC	vs. Base
Paths	15,954	13,121	6,588	**28,757**	**+80.25 %**
Valuations	13,295	8,633	4,827	**23,248**	**+74.86 %**
Exceptions	1,935	1,658	636	**7,080**	**+265.89 %**

COHEC explores more paths than Base on all ten libraries; the largest relative gains are `threetenbp` (+306.96 %), `jfreechart` (+247.77 %) and `sis-utility` (+214.38 %). JDart-2020 beats it on `ical4j` paths — reported, not buried — though COHEC produces more valuations and exceptional runs on that library.

RQ2 — against CONCOLLMIC, 30 stratified `commons-lang3` APIs, JaCoCo:

Metric	ConcoLLMic	COHEC	Change
Covered lines	355	405	+14.08 %
Covered instructions	1,829	2,004	+9.57 %
Covered branches	174	201	+15.52 %
Time (s)	11,167	3,894	–65.13 %
Total tokens	25,805,783	305,000	–98.82 %
Cost ($)	80.75	2.51	–96.89 %

RQ3 — the verifier's own statistics. 13,879 candidate models rejected across the ten libraries; bounded refinement recovers **9,267 (66.77 %)**. Recovery rate ranges from **27.93 %** on `xchart` to **90.34 %** on `threetenbp`. Compilation failure accounts for **over half** of all rejections — deep object graphs require correctly sequenced constructors, resolved overloads and respected access control, and one bad cast invalidates the candidate. `javac` diagnostics are precise, which is exactly why refinement works best there.

RQ4 — ablation, deltas on Paths / Valuations / Exceptions:

Variant	Paths	Valuations	Exceptions
w/o history retrieval	–4.49 %	–5.08 %	–12.30 %
w/o type constraints	–13.54 %	–15.78 %	–21.94 %
w/o heap state constraints	**–39.21 %**	**–40.38 %**	**–65.01 %**

Cost accounting is published rather than elided: HCS is invoked **11,047 times** consuming **93.0M tokens**, with tokens per accepted valuation ranging from **1,813** on `commons-lang3` to **14,563** on `xchart`.

Where the accounting is conservative

Three choices push results downward, and all three are stated. If the baseline times out on an API, that API is dropped for every configuration. If heap constraint solving fails on a path, the run falls back to the baseline result **while still being charged the tokens already spent** — so fallbacks count as baseline outcomes and reported gains are lower bounds. And RQ2's sample is the *least favourable* completed library: ConcoLLMic requires whole-library LLM instrumentation before per-API execution, three libraries were attempted at a total cost of $194.34, and only `commons-lang3` finished.

The model split is also disclosed rather than smoothed over: RQ1 uses DeepSeek-Chat v3.2 for cost reasons at 6,750-API scale, RQ2 uses Claude Sonnet 4.5 to match ConcoLLMic. Each comparison is internally controlled; the two are not cross-comparable on model strength, and the paper says so.

Stated limits

Every API is wrapped in a single-entry harness, so results may not transfer to other languages, applications with complex setup, or anything needing whole-program analysis. The ten subject libraries come from prior work and skew toward well-maintained utility-style code. The approach is explicitly weaker for long-range invariants, reflection, native code, concurrency, and environment-dependent behaviour outside the supported atoms — "our results demonstrate practicality for many library-style APIs, not general heap reasoning." `Exceptions` counts exceptional *executions*, not unique faults, and the construct-validity discussion says so directly rather than letting the number imply bug-finding.

What travels

The transferable pattern is the proposer/verifier split with an explicitly drawn trust boundary — which the authors themselves connect to CEGIS. What makes it work here is a property worth naming: the verifier is **cheap and total**. Compiling and running Java is fast relative to LLM inference, and it always returns an answer. Any domain with that property — synthesis against an executable spec, config generation against a schema validator, query generation against a type checker — can adopt the same structure and get a genuine soundness statement rather than a confidence score. The corollary matters just as much: where no such verifier exists, this architecture is unavailable, and you are back to trusting the model's output. The next paper in this report is what that looks like.

Closing line (verbatim). "Across 6,750 APIs from ten Java libraries, COHEC substantially improves over JDart and offers a better coverage–cost tradeoff than ConcoLLMic, demonstrating the effectiveness of verified heap model construction for object-sensitive behaviors."

1.2 · OSS SUPPLY CHAIN · Vulnerable packages grow 91.5%/yr, fixes flat at 1,650 — and the best LLM gatekeeper manages 62.8% F1👍😐💉

Vulnerability Evolution and the Promise of Automated Gatekeeping in Open-Source Software

Authors: Seyed Ali Akhavani (Northeastern University), Behzad Ousat, Selcuk Uluagac, Amin Kharraz (Florida International University). First two authors contributed equally.

Venue: RAID 2026 — International Symposium on Research in Attacks, Intrusions and Defenses.

The premise, stated in one line

"The scale of open-source adoption has outpaced our ability to secure it." The paper's contribution is that it makes this measurable across ecosystems rather than within one, and then asks what could plausibly close the gap.

Dataset

30,572 unified vulnerability reports affecting **16,802 packages**, spanning **10 languages** (C, C++, PHP, Rust, JavaScript, Go, Java, .NET, Python, Ruby) and **eight package managers** (PyPI, NPM, Packagist, Crates, Go, Maven, NuGet, RubyGems), **2017 to late 2025**, covering **467 distinct CWEs**.

Construction is worth reading closely because the filtering is unusually explicit. Two sources: the GitHub Advisory Database, restricted to *reviewed* advisories only — 24,500 records, deliberately excluding the 288K+ unreviewed ones — and Snyk, 23,709 reports. Unification normalizes to a nine-field schema. Duplicates are removed by CVE match, or where no CVE exists by (CWE, package, ecosystem, affected versions), prioritizing Snyk for richer metadata: **11,274 duplicates eliminated**. A further **6,363 reports were excluded** because their CWEs are flagged by MITRE as discouraged or prohibited (e.g. CWE-400, CWE-200) as too broad to act on. Historical package counts came from Wayback Machine snapshots of Libraries.io sampled twice yearly, because Libraries.io retains no history — a nice piece of measurement improvisation. Version numbers and publication dates were recovered for **16,618 unique packages**.

The paper places itself against prior work in a table: the largest comparable dataset it cites is 5,916 vulnerabilities over 958,547 packages in two ecosystems. This is roughly five times larger by report count and is the first to include C/C++ package vulnerabilities in this framing.

RQ1 — the remediation gap

Vulnerable packages grow at **91.54 % annually**; total packages at **26 %**. Fix counts rose from **467 in 2017 to 1,757 in 2022**, then plateaued at roughly **1,650 per year, 2022–2025**, with the absolute number bounded below ~2,000. Fix-to-report ratio settles near **0.44** and has stayed well under 1 since 2021. Between 2022 and 2024, ecosystems accumulated **over 7,000 unresolved vulnerabilities** — an average annual deficit around **2,500**.

Lifespan is defined formally in Appendix A.4, summing over all affected version ranges for a package:

∑i=1n(First Patched Version Release Datei−First Affected Version Release Datei)\sum_{i=1}^{n} \bigl(\text{First Patched Version Release Date}_i - \text{First Affected Version Release Date}_i\bigr)

where nn is the number of affected version ranges. Average lifespan rose from **1,215 days in 2017** to a peak of **1,988 days in 2022** (+63.7 %), then fell **25.7 %** to **1,475 days in 2025** — still **21 %** above the 2017 baseline. The authors group by *fix year* rather than introduction year specifically to avoid right-censoring bias, and they explain the 2025 decline mechanistically rather than optimistically: recent fixes concentrate on vulnerabilities introduced in 2024–2025, so older defects are being deprioritized rather than resolved, which drags the observed average down.

RQ2 — where the risk concentrates

Six CWEs account for **over 50 %** of all reports on average, and concentration varies sharply by ecosystem: NPM and Packagist are highly concentrated, Go and Crates much flatter. Universal patterns are CWE-79 (XSS), CWE-22 (path traversal) and CWE-94 (code injection). Ecosystem-specific ones track language semantics — CWE-1321 prototype pollution appears in **435** NPM reports; CWE-416 use-after-free and CWE-362 race conditions cluster in C/C++ and Crates.

The striking finding is CWE-506, embedded malicious code — deliberate injection rather than negligence. **7,398 reports**, up from **38 in 2018 to 2,952 in 2025**. Distribution: **NPM 6,679 (90.28 % of all CWE-506, and 67.24 % of NPM's entire CWE total)**, **PyPI 578 (7.81 %, 14.02 % of PyPI's total)**, everything else under 1 %. NPM and PyPI together exceed **98 %**. Two-thirds of everything reported in NPM is now deliberate attack rather than accidental bug.

Metadata analysis of all 7,398 characterizes the tradecraft: long names >10 characters (**75.6 %**), dashes (**71.7 %**), wildcard version targeting to maximize exposure (**49.7 %**) versus specific-version targeting (**27.3 %**). **11.4 %** use names closely resembling widely adopted libraries — one March 2024 campaign flooded PyPI with over 200 typosquats of packages like `tensorflow` and `pygame`. **15.6 %** use scoped names to impersonate internal corporate components, with **1,156 scoped packages** identified as dependency confusion. And NPM's lifecycle scripts (`preinstall`, `postinstall`) let attackers execute code before the package is ever imported.

RQ3 — LLMs as CI/CD gatekeepers

The evaluation design is the part worth borrowing. Rather than function-level snippets or whole-repository snapshots, the authors go **diff-centric**: for each advisory they resolve version ranges to four concrete releases (first patched, its predecessor, first vulnerable, its predecessor), extract changed files via the GitHub Compare API, and label the vulnerable version true with its CWE and the patched version false. That is two data points per pair, and it matches the moment a CI/CD gate would actually fire.

The honesty about noise is notable. Diffs have a **median of 900 changed lines** with a tail beyond 10,000, because coarse version jumps aggregate refactors and features alongside the fix — the authors call this an over-approximation of the true fixing change and cap the dataset at 5,000 changed lines (~32K tokens average). Final corpus: **582 version pairs → 1,164 data points, covering 1,434 advisories**, restricted to the five most prevalent CWEs per ecosystem (21 total) across 807 unique packages.

Ten models, self-hosted ones on four RTX 4090s:

Model	Precision	Recall	F1	Avg time (s)
GPT-5.1	63.9 %	61.8 %	**62.8 %**	2.50
GPT-4o	53.2 %	69.7 %	60.3 %	2.39

Model	Precision	Recall	F1	Avg time (s)
LLaMA-2:70B	45.8 %	**84.9 %**	59.5 %	6.81
DeepSeek-R1:70B	53.5 %	65.6 %	59.0 %	**49.90**
CodeGemma:7B	40.1 %	59.4 %	47.9 %	1.68
Qwen-2.5-Coder:32B	47.8 %	45.4 %	46.6 %	5.00
DeepSeek-Coder-V2:16B	47.7 %	37.3 %	41.9 %	4.46
Phi-3:14B	52.6 %	34.2 %	41.4 %	4.30
LLaMA-3.3:70B	39.9 %	34.4 %	36.9 %	9.55
Mixtral:8x7B	**90.1 %**	11.0 %	19.6 %	2.52

CodeLLaMA is excluded because it consistently refused to answer on safety grounds — reported rather than quietly dropped. Temperature and top-p variation had minimal effect.

Two secondary findings are more interesting than the headline. First, a **systematic prevalence bias toward CWE-79**: models misclassify injection-class and even orthogonal vulnerabilities as XSS, tracking training-set frequency rather than the code. Models with near-identical binary F1 differ substantially in categorical discrimination — CodeGemma:7B and Qwen-2.5-Coder:32B both sit at ≈0.47 F1, but Qwen shows a clean confusion- matrix diagonal while CodeGemma disperses. Even the best models confuse CWE-862/863 (authorization) and CWE-77/78 (command injection). Second, for **malicious-package detection**, diff analysis is structurally unsuitable — backdoors are present from the first release — so the authors switch to 600 confirmed malicious packages from the Datadog dataset and analyze source files directly: GPT-5.1 **83.5 %**, LLaMA-3.3 **83.3 %**, Qwen-2.5-Coder 74.4 %, Mixtral 72.9 %, CodeGemma **6.7 %**.

And a limit they raise themselves: supply-chain compromise arriving *through a dependency update* is invisible to code-level reasoning entirely. Their example is the axios incident, where malicious behaviour enters via an external dependency and nothing in the analyzed diff shows it. Prompt-based analysis cannot see that class of attack and must be augmented with threat intelligence.

The discussion section earns its place

Rust's Crates.io is the only ecosystem with a *negative* growth rate in vulnerable package counts, and the authors decline to attribute it purely to the language — noting lower popularity as a competing explanation. Go has the highest vulnerability growth but also the most visible community response, with the SafeText / SafeOpen / SafeArchive family targeting path traversal, which was among Go's top five CWEs. And on adoption: NPM package provenance launched over two years ago, yet examining NPM's monthly top-100 most-downloaded packages in February 2026, **only 9 %** had enabled it — including packages that appeared in provenance's own launch materials. The gap between tool availability and tool use is a finding in itself.

Read against the previous paper

"No single model resolves the trade-off between precision, recall, and latency." Compare the concolic paper, where an LLM is wrong on 13,879 candidates and it does not matter. The difference is not model quality — it is that heap-model construction has a total, cheap oracle and vulnerability triage does not. The gatekeeping proposal in this paper is best read as the correct response to that absence: a cascade,

fast high-recall first pass then slow high-precision confirmation, because in the absence of a verifier the only available move is to stage the uncertainty rather than discharge it. Mixtral's 90.1 % precision at 11.0 % recall is useless as a detector and quite reasonable as a confirmation stage — a point the paper makes explicitly.

The authors are also careful to separate detection from remediation, and put fixing outside scope rather than gesturing at it.

Closing line (verbatim). "These findings suggest that the future of OSS security cannot rely solely on manual review. Instead, we advocate for a hybrid approach of using LLMs to flag potential vulnerability introduction at the publication stage, while enforcing stricter limits on high-risk ecosystem features like installation hooks."

2.1 · BENCHMARKS · Curriculum plus ensemble lifts MCC from 0.44 to 0.73 — the data, not the model scale👍😐💉

Cross-Corpus Evaluation of Generalizable Vulnerability Detection in IoT Firmware

Authors: Sadib Hassan Rumman, Md. Shariful Islam, Md. Rayhanur Rahman

Venue: arXiv preprint 2608.11492 [cs.CR], 11 Aug 2026 (6 pages, 1 figure, 2 tables)

Link. arXiv:2608.11492. License: arXiv non-exclusive distribution (not CC), so no mirror. **Read at abstract level only** — the HTML full text was rate-limited during this run, so what follows is drawn from the abstract and metadata, and the methodology is not independently verified.

Worth carrying despite the depth limit, because the contribution is a measurement protocol rather than another detector. **IoTVulBench** is a human-verified benchmark for cross-corpus firmware vulnerability detection: IoTVulBench-Core is built from GitHub repositories, validated by three expert reviewers, and evaluated against a *contamination-screened held-out target* — the screening being the part that most work in this area skips.

The comparison is run across five model architectures, two tuning methods and three curriculum strategies, with ensemble, distillation and robustness analyses. Reported in MCC rather than accuracy or F1, which is the right choice for imbalanced detection: models trained on IoTVulBench reach **0.58**, against **0.44** for PrimeVul and **0.39** for D2A among matched single-source datasets. Staged curriculum learning lifts this to **0.69**; a diversity-optimized ensemble reaches **0.73** — **+0.42 MCC** over the strongest reference comparator (a static analyzer at **0.31**) and **+0.29** over PrimeVul. At a **0.5 % false-positive rate** the model misses **21 %** of vulnerabilities against **71 %** for the strongest comparator, which is the number that matters for anything deployed. Robustness: **86 %** of performance retained under identifier renaming.

The stated conclusion is that domain-matched training data and curriculum design, rather than model scale, drive generalization in firmware vulnerability detection. That claim is a useful counterweight in a literature that mostly reports architecture wins, and the low-FPR operating-point reporting is a habit more detection papers should adopt. Treat the numbers as the authors' claims pending a full read.

Cross-Paper Synthesis

The verifier is the variable, not the model. Both Outstanding papers put an LLM inside a security-relevant pipeline, and their outcomes diverge along one axis. The concolic paper has a downstream

oracle that is cheap, total and semantic: compile the initialization code, run it, analyze the resulting heap, compare against the proposed model. Because that oracle exists, the system can afford a proposer that fails 13,879 times, and can still make a soundness claim about every SAT it reports — the LLM is outside the trusted computing base by construction. The OSS gatekeeping paper has no such oracle. “Is this diff vulnerable?” has no executable checker, so model output *is* the answer, and the ceiling lands at 62.8 % F1 with a prevalence bias toward whatever CWE was common in training. The right generalization is not “LLMs work for program analysis but not for security” — it is that an LLM’s usable reliability in a pipeline is set by what sits downstream of it, and papers proposing LLM components should be read by asking what plays the verifier’s role.

Two papers, two ways of surviving a weak proposer. Given an unreliable generator you can either *discharge* the uncertainty (verify, and reject on inconclusiveness) or *stage* it (cascade a high-recall filter into a high-precision confirmer). COHEC does the first and gets a guarantee; the RAID paper recommends the second and gets a deployment strategy. Their model tables even suggest the roles: something like Mixtral’s 90.1 % precision at 11.0 % recall is exactly a confirmation stage, and LLaMA-2’s 84.9 % recall at 45.8 % precision is exactly a first pass. Neither is deployable alone. Worth noting that COHEC’s bounded refinement is quietly a hybrid of both — the verifier discharges, and the structured diagnostic routed back to the earliest affected stage is a staged retry that recovers two-thirds of failures.

Cost is becoming a first-class result. COHEC reports 93.0M tokens over 11,047 invocations, tokens per accepted valuation from 1,813 to 14,563, and a $2.51-vs-$80.75 comparison; the RAID paper reports total tokens and per-sample latency per model and flags DeepSeek-R1’s 50 seconds as disqualifying for CI/CD regardless of its 59.0 % F1. Both treat cost as a result rather than an implementation footnote, and in both cases it changes the conclusion — a method that is 5 % better and 85× more expensive is not better. A year ago this was rarely tabulated at all.

A gap nobody is standing in. The IoT benchmark paper reports its operating point at a **0.5 % false positive rate**, because a firmware scanner that fires constantly is worthless. The RAID paper evaluates ten models at their default operating points and finds precision between 39.9 % and 90.1 %. Nobody in the gatekeeping evaluation reports performance at a fixed low FPR — which is the only regime in which a CI/CD gate is deployable, since a gate that blocks half of all legitimate publishes will be switched off in a week. The two papers are one table apart from a genuinely useful joint result.

Writing & Rationale Insights

Name the bottleneck, then measure that it is one. The concolic paper could have asserted that heap constraints are the problem. Instead: a 60-API manual study showing 87.5 % completion but 42.5 % full branch coverage, with unresolved heap constraints identified as the dominant cause in the 45 % gap — followed by the benchmark-scale figure. Two measurements at two granularities, one qualitative and one aggregate. Structurally this converts the paper’s premise from a claim into a finding, before any contribution appears.

Publish the failure rate of your own component. RQ3 exists solely to report that 13,879 candidates were rejected. A weaker paper would have reported only the 66.77 % recovery, or omitted the section. Reporting the rejection count is what makes the trust boundary legible as *necessary* rather than decorative — and the follow-through in the discussion (“compilation failure accounts for over half”) turns an embarrassing number into a research direction. If your architecture contains a defensive component, the strongest evidence for it is a measurement of how often it fires.

Make conservative accounting explicit and cumulative. Three separate conservatism disclosures in the ASE paper: skipped APIs are dropped for every configuration; failed solving falls back to baseline while still being charged tokens; and the RQ2 sample is the least favourable of three attempted libraries. Each is small. Stated together they establish that reported gains are lower bounds, which is a much stronger claim than any single number. The reusable move is to *collect* your conservative choices into one paragraph rather than scattering them across method sections where a reader will never assemble them.

Define your metric in the appendix, formally, even when it sounds obvious. "Vulnerability lifespan" reads like a self-explanatory phrase. The RAID paper gives it a summation over all affected version ranges, which immediately reveals a decision a reader would otherwise have had to guess at — multi-range packages sum rather than take an envelope. One formula removes an entire class of reviewer objection about how the headline 1,988 days was computed.

Explain a favourable trend mechanistically instead of claiming it. Average lifespan fell 25.7 % from the 2022 peak. The easy reading is that the ecosystem is improving. The authors instead attribute it to fixes concentrating on recently introduced vulnerabilities while older ones go unremediated — a *worse* underlying story that better fits their own Figure 2. Volunteering the deflationary explanation for your most encouraging number is cheap credibility, and it pre-empts the reviewer who would have found it.

Report exclusions, including the awkward ones. The RAID evaluation notes that CodeLLaMA was dropped because it refused to answer on safety grounds. This is a mildly embarrassing detail with no upside to disclosing — and it is precisely why disclosing it works. Likewise, 6,363 reports were excluded for MITRE-discouraged CWEs and 11,274 as duplicates, both with the matching rule stated. A reader can reconstruct the dataset; more importantly, a reader believes the parts they cannot reconstruct.

State what your guarantee is relative to. The concolic paper says twice that soundness holds relative to the path condition *recorded by the underlying engine*, and that it does not recover heap constraints dropped during collection. Attaching the frame to the guarantee costs one sentence and stops the guarantee from being read as stronger than it is — which is what turns a correct claim into an overclaim in a reviewer's summary.